

GUI-Based Reed-Muller Decoder

Encoding, Error Simulation, and Step-by-Step Decoding Visualisation

Adwita Deshpande, Lakshay Khurana, Jash Gaglani

April 10, 2025

1 Introduction

Reed-Muller (RM) codes form a family of linear error-correcting codes with applications in digital communication, satellite transmission, and data storage. Their hierarchical structure and recursive construction make them an ideal candidate for systematic decoding strategies. This project presents a comprehensive Graphical User Interface (GUI) for interacting with Reed-Muller codes. The application facilitates encoding and decoding of messages, simulates channel noise, and visually demonstrates the stepwise decoding process.

The primary motivation is to create an interactive tool that supports educational exploration and practical demonstration of RM codes, especially for students and researchers in coding theory and digital communication.

2 Objectives

The objectives of the project are as follows:

- Develop a visually appealing and responsive GUI using `customtkinter`.
- Implement robust decoding using majority logic decoding with animated, step-by-step visual feedback.
- Allow users to simulate transmission errors by introducing single or multiple bit-flips.
- Provide clear visualization of generator matrices and intermediate decoding stages.

3 Features Implemented

3.1 Decoding

- Implements majority logic decoding suitable for RM codes.
- The decoder visualizes each monomial and its contribution to the reconstruction of the original message.

3.2 Error Simulation

- Users can introduce single or multiple bit errors.
- Errors can be placed randomly or manually at specified positions.

3.3 Matrix Visualization

- Generator matrix G is displayed for user-defined (r, m) values.

3.4 GUI Highlights

- Responsive layout with modular components.
- Real-time validation of user input (binary checks, length verification).
- Scrollable decoding output with explanations for each monomial and majority step.
- Error tracking and visual overlay during decoding.
- Export functionality for saving encoded/decoded results and matrices.

4 Architecture Overview

Frontend (GUI): Developed using `customtkinter`, enabling a modern aesthetic and clean widget design.

Backend: Encapsulation of logic in Python modules. Matrix operations are handled using NumPy for efficiency and scalability. Polynomial evaluation and monomial tracking are implemented to support general decoding.

Visualization: Uses Matplotlib for generator matrix rendering and additional graphical output.

5 Challenges Faced

- Generalizing the decoding logic to support any (r, m) configuration, including higher orders.
- Maintaining GUI responsiveness during intensive matrix operations and decoding steps.
- Designing an intuitive user experience for navigating complex decoding steps without overwhelming the user.

6 Conclusion

The GUI-based Reed-Muller decoder delivers a user-friendly and powerful environment for exploring binary error-correcting codes. Its stepwise decoding animation, matrix visualization, and error simulation features make it a practical tool for learning and demonstration. By blending intuitive design with theoretical depth, it bridges the gap between abstract concepts and practical understanding of RM codes.



Figure 1: Snapshot of the GUI during step-by-step decoding

7 Future Work

- Extend the tool to support encoding as well as decoding.
- Incorporate Reed-Solomon and BCH code modules for comparative study.
- Export all intermediate steps and results as a report or educational worksheet.
- Add support for real-time data transmission simulations (e.g., serial bit streams).